NLP Techniques to identify individuals that are at risk of self harm

J COMPONENT PROJECT REPORT

FALL 2023-2024

Submitted by
NIDA GHANI- 20BCE2850
TUSHAR SINGH - 20BCT0197
ARNAB MANDAL - 20BCE0026
MAYANK PURI - 20BDS0256
PULKIT SHARMA - 20BCE0615

in partial fulfillment for the award of the degree of

B. Tech

in

Computer Science and Engineering
Computer Science and Engineering with Specialization in Data Science
Computer Science and Engineering with Specialization in Information Security



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India

School of Computer Science and Engineering

Nov, 2023

DECLARATION

I hereby declare that the thesis entitled NLP Techniques to identify individuals that are at risk of self harm submitted by me, for the award of the degree of *Bachelor of Technology in Computer Science Engineering* to VIT is a record of bonafide work carried out by me under the supervision of Prof. Sathyaraj P..

I further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date : 17-11-2023

Signature of the Candidate

CONTENTS

Title

Abstract (300 words)

Introduction (Minimum of 1 and half pages)

Architecture diagram

Background study (Related papers and study) (All the papers)

Methodology (Explanation about algorithm, methods, datasets) (Minimum of 3 pages)

Proposed model (Diagram and Explanation) (Minimum of 2 pages)

Results and Discussion (Minimum of 7 pages)

Conclusion

References (Harvard style, sort by name)

Abstract

When it comes to mental health care, Natural Language Processing (NLP) approaches have opened up new possibilities by providing automated ways to identify suicidal ideation in digital text data. With social media sites like Twitter, where people freely express their ideas and feelings, this technology has great potential for early identification and treatment of serious mental health issues.

In this situation, the NLP approach known as TF-IDF (Term Frequency - Inverse Document Frequency) is quite effective. TF-IDF can identify potential signals of distress or crisis by comparing the frequency of particular phrases in Tweets connected to suicidal ideation against their occurrence in a wider context. With the help of this technique, it is possible to recognise patterns and language cues that may point to a person's propensity for suicidal thoughts, providing a proactive means of intervention and assistance.

Nonetheless, the importance of this study goes beyond suicidal thoughts alone. It's common for mental health illnesses to appear in subtle linguistic patterns on a variety of platforms, not just in debates about suicide. When combined with machine learning and deep learning methods, natural language processing (NLP) has the potential to identify minute linguistic indicators linked to a variety of mental health conditions, such as chronic anxiety or borderline personality disorder.

This research provides options for the creation of comprehensive tools that may identify and address a variety of mental health disorders found in digital life data by understanding the wider ramifications. These technologies have the ability to completely transform early detection, intervention, and support systems in the mental health care settings by giving people who might otherwise go unrecognised or underserved urgent help.

In the end, this multidisciplinary strategy at the nexus of NLP, machine learning, and mental health services represents a paradigm change towards the proactive use of technology for mental health support, promoting a more knowledgeable and caring digital ecosystem.

Introduction

Suicidal ideation is still a serious public health issue with far-reaching effects on both people and society as a whole. It is a clear sign of underlying mental disease. It can be difficult to spot these subtle signs of anxiety, and traditional screening techniques frequently miss them. Early identification and action are essential to preventing the terrible effects of suicide. The investigation of natural language processing (NLP) emerges as a powerful instrument to meet this pressing requirement in response to this urgency.

The project "NLP Techniques to Identify Individuals at Risk of Self-Harm" emphasises how important it is to use creative methods to spot suicidal thoughts in written expressions. The spread of online communication platforms in today's digital world—from social media hubs to forums and chat apps—has resulted in an unparalleled amount of textual data. This enormous language corpus provides a priceless chance to glean insights into people's mental health issues and even pinpoint those who may be at risk.

Using natural language processing (NLP) techniques provides an innovative way to gain access to this abundance of textual information. The overall goal is to create a proactive and effective system that can identify suicide ideation in its early stages by utilising complex algorithms that explore sentiment analysis, context understanding, and complex language patterns. This method goes beyond than just looking for overt allusions to self-harm or suicide; it also looks for more subtly suggestive language that may be used to indicate mental illness or emotional suffering.

The ultimate goal of this research is to build a strong framework that will allow for the identification of subtle signs of mental discomfort through analysis powered by natural language processing. Facilitating prompt intervention and the development of strong support networks is the goal. This proactive approach aims to close the gap in mental health care by quickly identifying those who are at risk and giving those who might otherwise go unnoticed or ignored urgent help and guidance.

Through the application of natural language processing (NLP) innovations to address a major social issue, this project represents a paradigm change in the approaches to mental health care. It has the ability to drastically change the paradigm of mental health care in addition to saving lives by providing timely intervention and support to individuals going through mental health crises. This technologically-driven proactive strategy represents a revolutionary step towards a more compassionate and functional mental health environment.

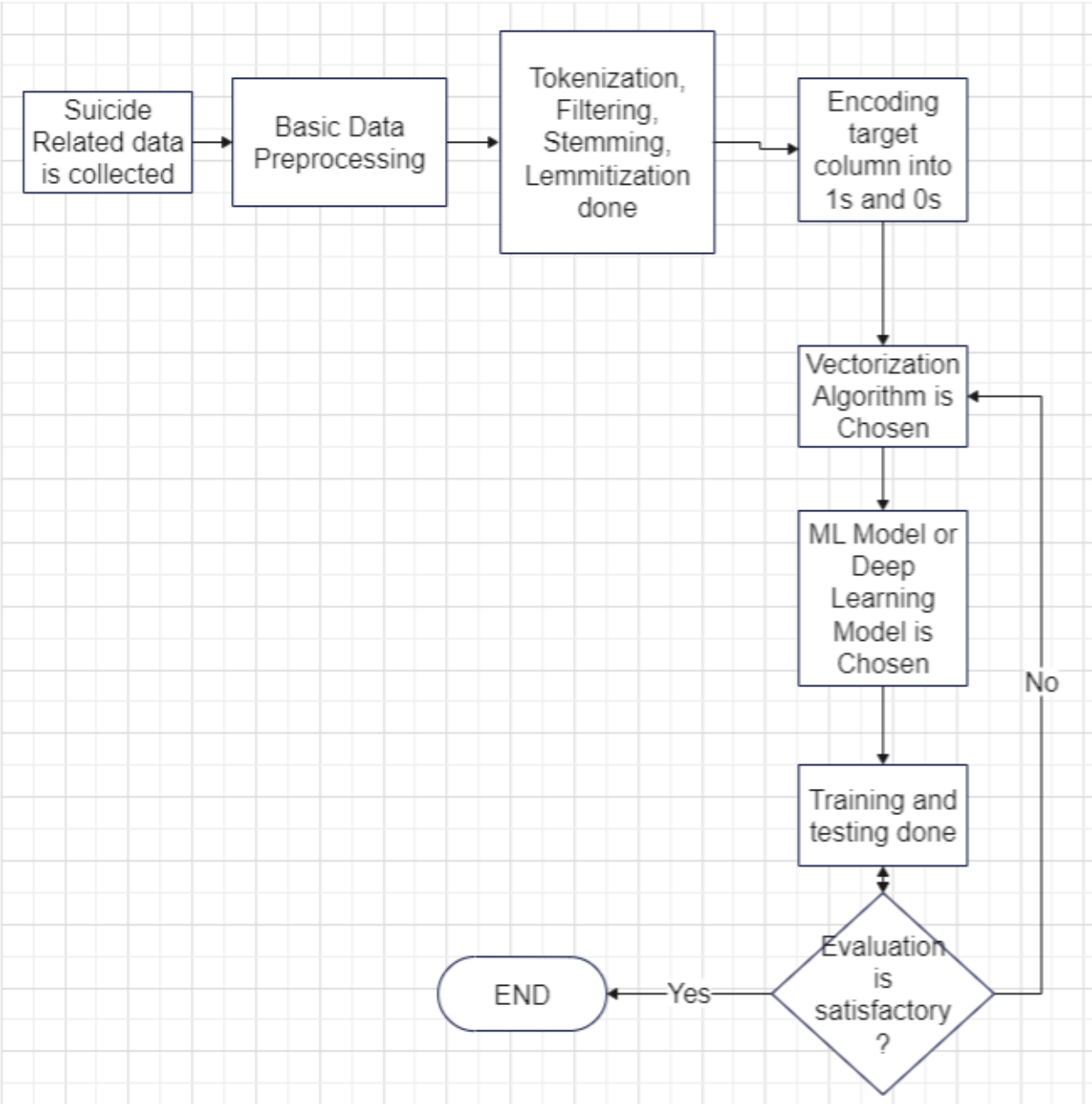
The algorithms used in this project will be a mixture of basic ML models, Deep Learning Techniques, various NLP Feature Extraction techniques and vectorization techniques to obtain the most accurate output.

The reason for using so many NLP techniques and models is to ensure the most accurate sentiment analysis that can be done for a cause of such a sensitive nature. Using a combination of models and techniques helps in achieving robust and reliable results. It also involves careful preprocessing of the text data, handling imbalances in the dataset, and possibly fine-tuning models to minimize false positives or false negatives.

Furthermore, the use of NLP approaches in mental health care goes beyond the treatment of suicidal ideation, demonstrating its ability to address a variety of mental health concerns. It sheds light on the way to an all-encompassing framework that can identify and assist people with a range of mental health issues, from chronic anxiety to borderline personality disorder.

This project, taken as a whole, represents a major step forward in the use of technology for mental health, welcoming a time when prompt intervention, compassion, and creativity come together to save lives and lessen the stigma associated with mental illness in our society.

Architecture Diagram



Background Study

Mayank Puri (20BDS0256) -

The World Wide Web and online social networks have expanded the network connecting people, allowing for the rapid dissemination of information to large numbers of people. There are several instances of this kind of online collaborative contagion, one of which is the development of self-destructive ideas in social media sites like Twitter, which has caused alarm. According to the study mentioned in **[1](Stacked CNN - LSTM approach for prediction of suicidal ideation on social media)**, the implications and findings of several machine classifiers that were applied to the point order of tweets and terms connected to suicide are discussed. The classifier can distinguish between more stressful information, such as suicidal creativity, other suicide-related topics, in-depth suicide-related facts, loyalty, campaign, and support. A simple classifier utilizing emotional, lexical, psychological, and structural characteristics from Twitter is used to link and identify allusions to suicide. This procedure makes use of clustering, bracketing, association rules, NLP (natural language processing), and numerous machine-learning techniques.

Finding and comprehending patterns of suicidal ideation represent a challenging task. Therefore, it is essential to develop a machine learning system for automated early detection of suicidal ideation or any abrupt changes in a user's behavior by analyzing his or her posts on social media. In **[2](Detecting and Analyzing Suicidal Ideation on Social Media Using Deep Learning and Machine Learning Models)**, we propose a methodology based on experimental research for building a suicidal ideation detection system using publicly available Reddit datasets, word-embedding approaches, such as TF-IDF and Word2Vec, for text representation, and hybrid deep learning and machine learning algorithms for classification. A convolutional neural network and Bidirectional long short-term memory (CNN–BiLSTM) model and the machine learning XGBoost model were used to classify social posts as suicidal or non-suicidal using textual and LIWC-22-based features by conducting two experiments. To assess the models' performance, we used the standard metrics of accuracy, precision, recall, and F1-scores.

Suicide is a critical issue in modern society. Early detection and prevention of suicide attempts should be addressed to save people's life. Current suicidal ideation detection (SID) methods include clinical methods based on the interaction between social workers or experts and the targeted individuals and machine learning techniques with feature engineering or deep learning for automatic detection based on online social contents. The study in **[3](Suicidal Ideation Detection: A Review of Machine Learning Methods and Applications)** is the first survey that comprehensively introduces and discusses the methods from these categories. Domain-specific applications of SID are reviewed according to their data sources, i.e., questionnaires, electronic health records, suicide notes, and online user content.

Suicide has become a serious problem, and how to prevent suicide has become a very important research topic. Social media provides an ideal platform for monitoring suicidal ideation. A study

in **[4](Detecting Suicidal Ideation in Social Media: An Ensemble Method Based on Feature Fusion)** presents an integrated model for multidimensional information fusion. By integrating the best classification models determined by single and multiple features, different feature information is combined to better identify suicidal posts in online social media. This approach was assessed with a dataset formed from 40,222 posts annotated by Weibo. By integrating the best classification model of single features and multidimensional features, the proposed model ((BSC + RFS)-fs, WEC-fs) achieved 80.61% accuracy and a 79.20% F1-score.

According to an estimate of World Health Organization, each year approximately 700,000 people die by suicide, with many more contemplating suicide. Early detection of suicidal ideation and proper treatment are two of the most effective techniques to preventing suicide attempts. People who are depressed or suicidal are increasingly using social media to express themselves. The main aim of the research in **[5](Suicide ideation detection from online social media: A multi-modal feature based technique)** is to provide early detection of suicide ideation by evaluating online social media. A well-labeled dataset of suicide thoughts was created on Reddit and Twitter and six feature groups were identified that included not just clinical suicidal symptoms but also online behaviors on social media. A multimodal model is proposed using these feature groups for identifying suicidal thoughts on social media. An accuracy of 87% was obtained using the Logistic regression classifier which outperforms other baselines.

Nida Ghani (20BCE2850)-

The paper mentioned in **[6](A review of natural language processing in the identification of suicidal behavior)** reviews various studies on the use of Natural Language Processing (NLP) to detect suicidal ideation and behaviors from verbal expressions. It examines 31 selected studies from a pool of 9,711 candidates, primarily from English-language sources. The paper finds that NLP methods have shown strong potential in accurately identifying individuals at risk of suicide. However, the paper notes limitations such as non-standardized reporting, small sample sizes, and selection biases. The paper recommends further research to understand NLP's dissemination and implementation, aiming to improve suicide prevention efforts and reduce global suffering.

This study done in **[7](A Comparative Analysis on Suicidal Ideation Detection Using NLP, Machine, and Deep Learning)** conducts a comparative analysis of Machine Learning (ML) and Deep Learning (DL) models for detecting suicidal ideation in Twitter data. The study aims for accurate early detection and finds that a BiLSTM DL model performs the best, achieving 93.6% accuracy. The research bridges psychology and computer science and proposes future work in feature extraction techniques, imbalanced datasets, and real-world applications for mental health professionals.

Due to the increasing awareness of mental health issues nowadays, the paper mentioned in **[8](Natural language processing applied to mental illness detection: a narrative review)**

takes a step towards assessing it's impact on society's health and well being. The study surveys a total of 399 different papers from 10,467 records. The data was taken from Social media posts, Magazines, Electronic health records, Interviews and screening surveys. It portrays that 59% of methods used fot mental illness detection use basic ML methods such as Decision tree, adaboosting etc. In Deep learning methods, CNN, RNN, Transformer based methods (including BERT, Roberta and XLNET) and other hybrid methods (CNN-LSTM) were surveyed. The final conclusion was that most of the methods used traditional supervised machine learning models that can sometimes be unstable. A need for more complex models is imminent.

The detection of mental disorders and suicidal ideation from social media at an early stage provides a basis of intervention that could be potentially life saving. The study mentioned in **[9](Suicidal Ideation and Mental Disorder Detection with Attentive Relation Networks)**, portrays text representation with lexicon-based sentiment scores and proposes the use of a relation network to detect suicidal ideation with related risk indicators. The relation module is further equipped with the attention mechanism to prioritize more critical relational features. Through experiments on three real-world datasets, the model gives very accurate results. Essentially, the integration of suicidal ideation with sentimental indicators leads to the development of RNs with an attention mechanism for relational encoding.

Understanding the importance of traditional ML models is essential to build more complex models that result in a higher accuracy. In the study mentioned in **[10](A New Approach To Suicide Ideation Detection from Text Content),** a novel approach to detect suicidal ideation using NLP and machine learning methods was explored. Three classifiers, RF, SVM, and NB, were employed and different feature sets, including Statistical, TFIDF, POS, N-grams, and Topic Modeling features, were extracted. Feature reduction techniques such as PCA and IG were applied to improve performance. The findings demonstrated that using expressive feature sets and selecting relevant and informative features resulted in a more accurate classification process, with the RF classifier achieving the highest accuracy of 97.02% when PCA was utilized as the feature reduction technique. So, PCA stands to be very beneficial for the overall model.

A recent innovation called MindWatch **[11](MindWatch: A Smart Cloud-based AI solution for Suicide Ideation Detection leveraging Large Language Models)** was recently created as a tool that can be used by mental health professionals for early detection of suicidial ideation and personalized psychoeducation. It employed ALBERT and Bio-Clinical BERT language models, fine-tuning them with the Reddit dataset to build classifiers. These models were compared to bi-LSTM, ALBERT, Bio-Clinical BERT, OpenAI GPT3.5 (using prompt engineering), and an ensemble voting classifier for suicide ideation detection. Moreover, personalized psychoeducation was achieved with the Llama 2 foundation model, using prompt engineering as a state-of-the-art method. All the models had an accuracy, precision and recall that were greater than 92%. It highlights the importance and usefulness of Large Language Models and

Transformer models because of it's ability to be tailored according to the user's specific needs and requirements. This model was developed with the help of the AWS cloud.

With the growing importance of Large Language Models, it is only fitting to expand its use into the field of Mental Health. The study mentioned in **[12] (Evaluation of ChatGPT for NLP-based Mental Health Applications),** evaluates the accuracy of ChatGPTs zero-shot classification. In this work, LLM-based ChatGPT was utilized with gpt-3.5-turbo backend to classify text-based social media posts into three mental health categories: stress, depression, and suicidality. F1 scores of 0.73, 0.86, and 0.37 were obtained for stress detection, depression detection, and suicidality detection, respectively. The social media posts were annotated with labels for classification, and the ChatGPT API performed the classification using an input prompt. A baseline model that always predicted the dominant class and has an F1 score of 0.35, 0.60 and 0.19 was used for comparison and clearly, the Large Language Model prevailed in this aspect. Hence, it can be further understood that LLMs stand a good chance in suicidal ideation detection.

Arnab Mandal(20BCE0026)

This shows[13] about anxiety and sadness which are examples of mental health issues that can have extremely dangerous repercussions. One less well-known sign of depression that is primarily seen in young individuals is self-harm. Writing style has been found to be a good indicator of mental health risks and well-being, and social media offers a wealth of user-generated content for researchers to examine. Early discovery is vital for issues related to mental health. It was suggested to use the shared task eRisk in this situation. This work explains the NLP-UNED group's involvement in the 2021 T2 subtask. The task assigned to participants was to develop systems that could identify Reddit users who were at risk of self-harm. With features derived from text data, TF-IDF keywords, first-person pronoun usage, sentiment analysis, and self-harm, we suggest a feature-driven classifier.

It explores[14] on precisely identifying self-harm presentations to emergency departments (EDs) can facilitate the provision of mental health services in a timely manner, assist in assessing the prevalence of suicidal intent within a community, and support the assessment of the results of

public health programs aimed at preventing suicide. Our goal is to create an automated system that can identify instances of self-harm in ED triage notes, as there is currently a deficiency in manual self-harm reporting in the department.

It[15] is imperative for text-based counseling services to identify users who pose a danger of suicide in order to prioritize and flag at-risk clients. The goal of this research is to create a domain knowledge-aware risk assessment (KARA) model that will enhance our capacity to identify suicide in online counseling environments. An emotional support system set up in Hong Kong provided us with the biggest known de-identified dataset, which consisted of 5682 Cantonese talks between counselors and help-seekers. Of those, 682 talks revealed suicidal thoughts during a crisis. We created a computer-processable network called the suicide-knowledge graph to capture domain knowledge linked to suicide. The integration of this knowledge network enhanced the capacity of a deep learning algorithm to recognize individuals seeking assistance during a crisis. The identical task was run via a typical NLP model as a baseline.

Public health prioritizes[16] preventing suicide, yet little is known about the risk factors associated with suicide following a medical hospitalization. Women are particularly affected by this issue, as their suicide rates in the US are rising at a disproportionate rate. To assess the difference in the risk of self-harm and suicide attempt among women with depression, bipolar illness, and persistent psychosis after being admitted to a general medical hospital. Using electronic health record data from 1628 women at the University of California Los Angeles Integrated Clinical and Research Data Repository, we constructed a machine learning algorithm that detected risk variables of suicide attempt and self-harm following general hospitalization. We used the algorithm on a bigger sample of 140,848 women from the New York City Clinical Data Research Network in order to evaluate replicability.

In this project,[17] two academic medical centers with radically dissimilar healthcare systems—King's College London (KCL), in the UK, and Weill Cornell Medicine (WCM), in the US—created NLP techniques independently. We then shared these ideas using a mechanism we developed. We used manually annotated clinical notes (KCL: n = 4,911 and WCM = 837) to assess and compare the algorithms' performance. Our quantitative evaluation found that, despite

the NLP approaches' successful technical porting, independently developed NLP approaches do not have the same level of success as those at the institution where the NLP algorithm was developed. However, they can still detect suicidality at another healthcare organization with a different EHR system, clinical documentation processes, and culture.

The paper[18] was carried out in accordance with the PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-analyses) standards to increase the level of transparency. The paper uses NLP techniques like text mining or text analytics and actively looks for keywords that might point at a hint like "mental health" OR disorder OR depression OR suicide OR psychotic OR psychiatry OR "self-harm" OR suicidal etc. The paper used datasets from PubMed, MEDLINE, Embase, PsycINFO, and Global Health through the OVID platform. Several papers discovered in the reference lists of the included studies were also included.

Free-form verbal expression[19] in written and spoken form can be used to identify a variety of psychological problems using natural language processing (NLP) techniques. These technologies could be very helpful for identifying suicide risk and perhaps reducing world suffering. These techniques effectively enable for considerably faster computing of data at levels that people frequently find difficult to comprehend, which in certain situations results in better diagnostic accuracy and/or prognosis treatment success prediction.

TUSHAR SINGH (20BCT0197)

This study [20]focused on uncovering connections between intimate partner violence (IPV) and suicide by designing a tool to identify IPV-related mentions in single suicide cases reported in the National Violent Death Reporting System (NVDRS). The researchers examined a dataset of 10,000 cases spanning from 2010 to 2018, employing machine learning techniques to develop and validate the tool. The results were encouraging, showing the tool's effectiveness in pinpointing IPV circumstances. This development is crucial for enhancing our comprehension of the role of IPV in suicides and enables more targeted efforts for prevention.

This paper [21] explores the relationship between intimate partner violence (IPV) and suicide, utilizing data from the National Violent Death Reporting System (NVDRS). By developing a tool to identify IPV references in single suicide cases, the researchers achieved promising results. This tool holds great promise for informing future research and guiding the development of targeted prevention strategies for suicides linked to IPV.

This study [22] presents the development of a domain knowledge-aware risk assessment (KARA) model to enhance suicide detection in online counseling services. The KARA model, informed by a suicide-knowledge graph, outperforms standard natural language processing (NLP) models in identifying individuals in crisis. Results show higher precision and recall for both non-crisis and crisis cases, indicating its clinical relevance and potential for improving suicide prevention in online counseling. This approach addresses the critical need to promptly identify at-risk individuals in remote counseling settings.

The paper[23] employed machine learning and natural language processing to predict depression-related posts on Reddit, particularly those related to suicidal ideation in subreddits like r/depression and r/SuicideWatch. By using various machine learning algorithms, we aimed to identify individuals at risk of depression. Results showed promising accuracy and f1-scores, with Logistic Regression achieving 77.29% accuracy and a 0.77 f1-score, among others. This approach complements traditional mental health assessment methods and offers valuable insights into at-risk individuals.

This study[24] examines mental health issues, like depression, self-harm, and suicide, and their clinical diagnosis challenges due to stigma. It explores using deep learning for non-clinical mental health assessment through online social media content. The systematic literature review of 96 studies highlights the growing trend in deep learning for prediction. The research emphasizes the interconnected nature of these issues, the shift to virtual communication, and the role of AI. It provides insights into datasets and proposes a deep learning framework, contributing to non-clinical mental health assessment and future research.

This paper[25] focuses on detecting depression and suicidal tendencies in social media users. With depression affecting many and exacerbated by the COVID-19 pandemic, early identification is crucial. The study proposes a classifier that combines Natural Language Processing (NLP) and machine learning techniques to analyze users' tweets, achieving a 75% accuracy rate in recognizing these tendencies. This hybrid approach offers the potential for early intervention and support on social media, emphasizing the importance of using digital platforms for mental health monitoring in a global mental health crisis.

Pulkit Sharma (20BCE0615)

The paper [26] was about suicides in adolescents. A long-term and short-term cyclic neural network was used to filter the redundant data and vector weights were used to further analyse the psychological behaviour. The network's architecture was such that it was able to handle the time series data eliminating the uneccesary info further increasing the accuracy. Network used the mechanisms like forgetting gates,input gates and output gates. The paper demonstrated effective recognition techniques.Overall the paper provides a valuable insight into understanding and addressing suicidal tendencies in adolescents.

Suicidal thoughts can emerge as a cause of mental stress. In [27], the paper says that the traditional method of suicide prediction uses the psychological stress and associates it to suicide possibility, and there is minor difference in the predictions. To advance the study 2 ML algorithms were used: Random Forest and Support Vector Machine. The messaging data was analysed to compare the algorithms.After comparison, the Random Forest had more accuracy than Support Vector Machine.

[28] In this study, 10,000 Reddit user posts were used which was available as a dataset at Kaggle, the dataset was evenly split with 5000 suicidal and 5000 non-suicidal posts. Then 5 ML algorithms were used to perform binary classification of data these were: Multinominal Naïve Bayes, Logistic Regression, Support Vector Machine, Random Forest, Long term Short Memory. As a result, Support Vector was most Accurate with 91.3% accuracy

[29] In this paper, the existing ML techniques for suicide ideation have been compared on broadly two categories: Text Based indicators and Behaviour based indicators. The algorithms that were compared were: TF-IDF, N-gram, LIWC, Naïve Bayes, KNN, SVM, Random Forest, XGBoost, Logistic Regression, and CNN. As a result, Logistic regression had the highest accuracy and XGBoost had highest average accuracy across various datasets which was over 80%.

[30] This paper emphasises the lack of accuracy in the prediction of suicides through text data, the data for support is also predicted as suicidal in the traditional methods, and the severity levels are also misinterpreted. So in this paper, a bidirectional RNN(GRU and LSTM) model is used for emotion and polarity clarity while analyzing the data. The result shows that LSTM has an increased accuracy of 8% which takes it to the highest accuracy in existing prediction systems and techniques.

[31] Paper emphasis the importance of early detection of suicidal tendencies on social media. Here an ANN is used for the same. The ANN has 3 layers where it used Word2Vec and Doc2Vec

for data embedding and TF-IDF for data tokenisation and vectorisation connecting to a hidden layer of 8 nodes. The ANN was implemented in Python and the dataset was split into 80% for training and 20% for testing.Performance of ANN with loss function algorithm was evaluated.Average accuracy was 89% and after training it was 91%. Average loss was 26% and after training it was 16%. At the end, ANN was also compared with AVM and ANN clearly was more accurate.

[32] In this paper, Four Machine Learning Algorithms were used and compared using 10-fold cross-validation for suicide ideation. These Algorithms were: Decision Tree, Support Vector Machine, Naïve Bayes, and Logistic Regression. The average cross-validation score of the algorithm is used to evaluate the performance. ML-GWO was used to further improve the performance and overcome the drawbacks of ML models. As a result Decision Tree had the highest accuracy of 97%.

Methodology

The purpose of our project is to test and demonstrate various NLP techniques to detect suicidal ideation from the content that is posted online on various social media platforms.

In order to execute this, the following process flow has been executed:

1. **Data Collection:**

   The data has been collected from the postings of various reddit users.

2. **Data Pre-processing:**

   In the project, several essential data preprocessing steps have been applied to optimize the text data for machine learning models, particularly for the task of detecting suicidal ideation.

   The initial dataset, represented as a Pandas DataFrame, contains columns such as 'ID,' 'text,' and 'class,' with the latter indicating whether the text is associated with suicidal thoughts ('suicide') or not ('non-suicide').

   To ensure data integrity, the 'Unnamed' column has been renamed to 'ID,' and rows with missing values in the 'class' column have been removed.

   Following this, a custom cleaning function was implemented to remove non-alphabetic characters and extraneous whitespaces from the 'text' column.

   Finally, character encoding was done to convert the target column (“class”) into 0s (“non-suicidal”) and 1s (“suicidal).

	ID	text	class
0	2	Ex Wife Threatening SuicideRecently I left my ...	suicide
1	3	Am I weird I don't get affected by compliments...	non-suicide
2	4	Finally 2020 is almost over... So I can never ...	non-suicide
3	8	i need helpjust help me im crying so hard	suicide
4	9	I'm so lostHello, my name is Adam (16) and I'v...	suicide

3. **NLP techniques to further process the text:**
   In the project, the Natural Language Toolkit (nltk) played a crucial role in the data

preprocessing pipeline. After cleaning the text data by removing non-alphabetic characters and extraneous whitespaces, the nltk library was employed for advanced text processing tasks. Specifically, the data underwent tokenization, which involves breaking down the sentences into individual words. This tokenized representation allowed for a granular analysis of the textual content, treating each word as a separate entity for subsequent processing.

Additionally, the preprocessing involved lowering the case of all words in the tokenized text, a step known as lowercasing. This standardization ensures that words are treated uniformly regardless of their original case, avoiding redundancy in the model's learning process.

Furthermore, stopwords, commonly occurring words in the English language with limited semantic value, were removed from the tokenized and lowercased text. This step is essential for focusing the model on more informative words that contribute to the context and meaning of the text, rather than common, less informative terms.

The final preprocessing step involved lemmatization using the WordNet lemmatizer provided by nltk. Lemmatization reduces words to their base or root form, ensuring that different inflections of a word are represented consistently. This step is particularly beneficial for capturing the core meaning of words and reducing feature dimensionality, ultimately enhancing the model's ability to discern patterns related to suicidal ideation.

In summary, the combination of cleaning, tokenization, lowercasing, stopword removal, and lemmatization using nltk contributed to a refined and standardized text representation, laying a solid foundation for subsequent machine learning tasks in the detection of suicidal ideation within the dataset.

	ID	text	class	cleaned_text	tokenized_text	lowercased_text	filtered_text	lemmatized_text	class_binary
0	2	Ex Wife Threatening SuicideRecently I left my ...	suicide	Ex Wife Threatening SuicideRecently I left my ...	[Ex, Wife, Threatening, SuicideRecently, I, le...	[ex, wife, threatening, suiciderecently, i, le...	[ex, wife, threatening, suiciderecently, left,...	[ex, wife, threatening, suiciderecently, left,...	1
1	3	Am I weird I don't get affected by compliments...	non-suicide	Am I weird I dont get affected by compliments ...	[Am, I, weird, I, dont, get, affected, by, com...	[am, i, weird, i, dont, get, affected, by, com...	[weird, dont, get, affected, compliments, comi...	[weird, dont, get, affected, compliment, comin...	0
2	4	Finally 2020 is almost over... So I can never ...	non-suicide	Finally is almost over So I can never hear has...	[Finally, is, almost, over, So, I, can, never,...	[finally, is, almost, over, so, i, can, never,...	[finally, almost, never, hear, bad, year, ever...	[finally, almost, never, hear, bad, year, ever...	0
3	8	i need helpjust help me im crying so hard	suicide	i need helpjust help me im crying so hard	[i, need, helpjust, help, me, im, crying, so, ...	[i, need, helpjust, help, me, im, crying, so, ...	[need, helpjust, help, im, crying, hard]	[need, helpjust, help, im, cry, hard]	1
4	9	I'm so lostHello, my name is Adam (16) and I'v...	suicide	Im so lostHello my name is Adam and Ive been s...	[Im, so, lostHello, my, name, is, Adam, and, I...	[im, so, losthello, my, name, is, adam, and, i...	[im, losthello, name, adam, ive, struggling, y...	[im, losthello, name, adam, ive, struggling, y...	1

4. Vectorization and Embeddings

Vectorization and embeddings play a pivotal role in Natural Language Processing (NLP) by bridging the semantic gap between human language and machine learning algorithms. In NLP, words are inherently qualitative, and their meanings are context-dependent. Vectorization, the process of converting textual data into numerical vectors, provides a quantifiable representation that algorithms can comprehend.

Embeddings take this idea further by capturing semantic relationships between words in a continuous vector space.

In this project, we have explored the following vectorization and Embedding Algorithms:

TF-IDF (Term Frequency-Inverse Document Frequency):

1) The steps to the TF-IDF algorithm are as presented below:
   Term Frequency (TF):For each term in a document, calculate its frequency within that document. This step captures the importance of each term within the document.
2) **Inverse Document Frequency (IDF):** Measure the uniqueness or importance of each term across the entire corpus. Terms that are frequent across all documents receive a lower IDF score.
3) **TF-IDF Calculation:** Multiply the TF and IDF values to obtain the TF-IDF score for each term in each document. This step assigns a weight to each term based on its frequency in the document and rarity across the corpus.

Formula:

$$\text{TF-IDF}(t,d,D)=\text{TF}(t,d)\times\text{IDF}(t,D).$$

4) **Feature Matrix:** Organize the TF-IDF values for each term in each document into a feature matrix. This matrix serves as the input for machine learning models.
5) **Model Training and Prediction:** Utilize the TF-IDF feature matrix as input for machine learning models

	ID	text	class	cleaned_text	tokenized_text	lowercased_text	filtered_text	lemmatized_text	class_binary	processed_text	...	youtuber	youve	yr
0	2	Ex Wife Threatening SuicideRecently I left my ...	suicide	Ex Wife Threatening SuicideRecently I left my ...	[Ex, Wife, Threatening, SuicideRecently, I, le...	[ex, wife, threatening, suiciderecently, i, le...	[ex, wife, threatening, suiciderecently, left,...	[ex, wife, threatening, suiciderecently, left,...	1	ex wife threatening suiciderecently left wife ...	...	0.0	0.0	0.0
1	3	Am I weird I don't get affected by compliments...	non-suicide	Am I weird I dont get affected by compliments ...	[Am, I, weird, I, dont, get, affected, by, com...	[am, i, weird, i, dont, get, affected, by, com...	[weird, dont, get, affected, compliments, comi...	[weird, dont, get, affected, compliment, comin...	0	weird dont get affected compliment coming some...	...	0.0	0.0	0.0
2	4	Finally 2020 is almost over... So I can never ...	non-suicide	Finally is almost over So I can never hear has...	[Finally, is, almost, over, So, I, can, never,...	[finally, is, almost, over, so, i, can, never,...	[finally, almost, never, hear, bad, year, ever...	[finally, almost, never, hear, bad, year, ever...	0	finally almost never hear bad year ever swear ...	...	0.0	0.0	0.0
3	8	i need helpjust help me im crying so hard	suicide	i need helpjust help me im crying so hard	[i, need, helpjust, help, me, im, crying, so, ...	[i, need, helpjust, help, me, im, crying, so, ...	[need, helpjust, help, im, crying, hard]	[need, helpjust, help, im, cry, hard]	1	need helpjust help im cry hard	...	0.0	0.0	0.0
4	9	I'm so lostHello, my name is Adam (16) and I'v...	suicide	Im so lostHello my name is Adam and Ive been s...	[Im, so, lostHello, my, name, is, Adam, and, I...	[im, so, losthello, my, name, is, adam, and, i...	[im, losthello, name, adam, ive, struggling, y...	[im, losthello, name, adam, ive, struggling, y...	1	im losthello name adam ive struggling year im ...	...	0.0	0.0	0.0

5 rows × 5010 columns

Word2Vec embedding algorithm:

The steps to the Word2Vec algorithm are as presented below:

1) **Building Vocabulary:** Create a vocabulary by collecting all unique words from the entire corpus. Assign a unique integer index to each word in the vocabulary.
2) **Model Training:** Train the Word2Vec model using the selected architecture. The model adjusts word vectors during training to accurately predict surrounding words or the target word based on its context.
3) **Vector Representations:** After training, each word in the vocabulary is represented as a dense vector in the learned vector space. These vectors capture semantic relationships and similarities between words.
4) **Word Similarity:** Use the cosine similarity or other distance metrics to measure the similarity between word vectors. Similar words will have vectors that are close to each other in the vector space.
5) **Application to Downstream Tasks:** Utilize the learned word embeddings as features for downstream machine learning tasks such as sentiment analysis, named entity recognition, or any task requiring understanding of word semantics.

Doc2Vec embedding algorithm:

Doc2Vec is an algorithm that extends the Word2Vec model to learn document-level embeddings. Here are the steps in Doc2Vec:

1) **Document Tagging:** Assign a unique tag or label to each document in the corpus. This tag acts as an identifier for the document during the training process.

2) **Tokenization:** Tokenize each document into individual words or phrases. This creates the input data for the model.
3) **Building Vocabulary:** Create a vocabulary by collecting all unique words and phrases from the entire corpus. Each word or phrase is associated with a unique integer index.
4) **Model Architecture:** Construct a neural network model where each document is represented by a unique vector, and words are also represented as vectors. The model learns to predict words based on their context within documents.
5) **Training the Model:** Train the Doc2Vec model by feeding document-word pairs to the neural network. The model adjusts the vectors during training to accurately predict words in the context of their associated documents.
6) **Generating Document Embeddings:** Once trained, the model can be used to infer embeddings for new documents. These embeddings capture the semantic information of the entire document.
7) **Similarity Comparison:** Measure document similarity by comparing the cosine similarity between the vectors representing two documents. Similar documents will have higher cosine similarity scores.
8) **Application to Downstream Tasks:** Use the learned document embeddings as features for downstream machine learning tasks such as classification, clustering, or information retrieval.

The word cloud for the vectorized data:



*Some innapropriate words have been stricken out in the above depiction

1. **5. Choosing the right ML and Deep-Learning Models:**

   For our project we have experimented with various ML and Deep-Learning models that have been applied to all the different vectorization and embedding algorithms.

   These are the following Machine Learning Models that have been Used to train and test the data obtained from the vectorization and embedding algorithms:

   TF-IDF:

   - **Decision Tree:** A tree-like model that makes decisions based on features, splitting data into branches and leaves for classification or regression.
   - **Random Forest:** An ensemble learning method that builds multiple decision trees and combines their predictions to improve accuracy and control overfitting.
   - **logistic regression:** A linear model used for binary or multiclass classification, estimating the probability of an instance belonging to a particular class.
   - **An ensemble algorithm that uses votingclassifier:** A linear model used for binary or multiclass classification, estimating the probability of an instance belonging to a particular class.
   - **Recurrent Neural Network:** A type of neural network designed for sequence data, allowing information persistence and learning from sequential patterns.
   - **Bi-Directional Long Short Term Memory Neural Network:** An extension of the LSTM architecture, capable of capturing dependencies in both forward and backward directions for improved sequence modeling.
   - **Convolution Neural Network:** A neural network architecture particularly effective for image recognition, using convolutional layers to detect hierarchical patterns.

   Word2Vec:

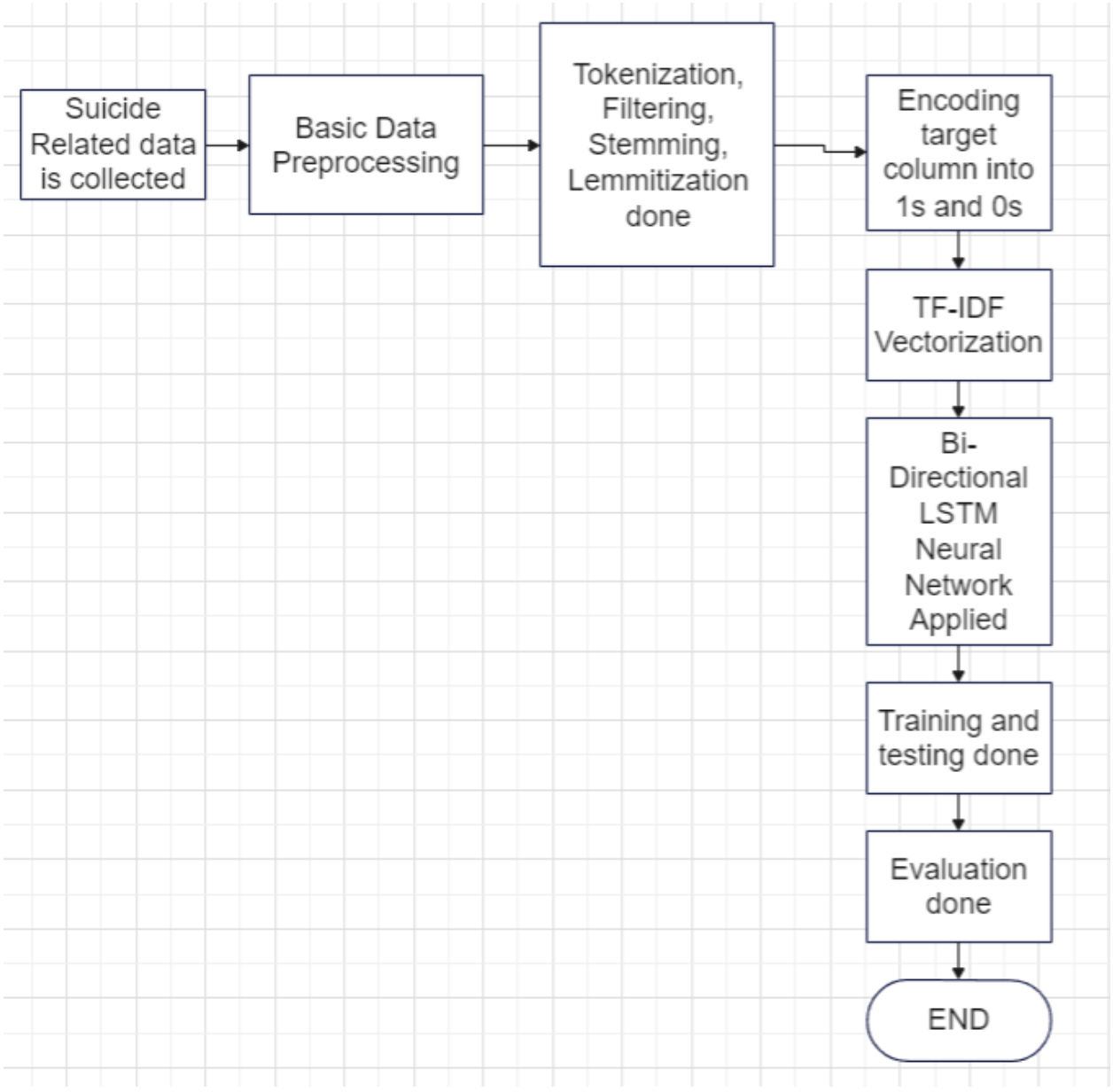
   - **Logistic Regression:** Applying logistic regression to classify or predict outcomes based on learned Word2Vec embeddings.
   - **Recurrent Neural Network:** Using a recurrent neural network with Word2Vec embeddings to model sequential dependencies in text data.

   Doc2Vec:

   - **logistic regression:** Applying logistic regression to classify or predict outcomes based on learned Doc2Vec embeddings.
   - **Recurrent Neural Network:** Leveraging a recurrent neural network with Doc2Vec embeddings to capture sequential patterns in document-level data.

Proposed Model

After doing a thorough analysis on a variety of different models, our proposed model is A **Bi-directional LSTM that utilizes the processed text from TF-IDF vectorization** to implement sentiment analysis.



The steps followed are as given below:

First, preprocessing of the text data was done to ensure its suitability for effective analysis. This involved a thorough cleaning process where we removed non-alphabetic characters and eliminated unnecessary whitespaces to create a standardized and consistent text corpus. Leveraging the Natural Language Toolkit (nltk), we performed tokenization, breaking down the text into individual words. This was followed by converting all words to lowercase, a critical step to ensure uniformity in subsequent analyses. Further refinement was achieved through stopword removal and lemmatization, actions that helped reduce noise and transform words to their base forms, respectively.

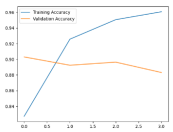
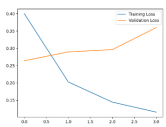
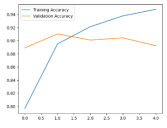
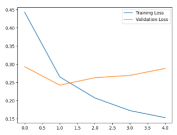
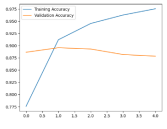
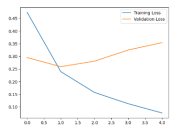
Next, we employed the TF-IDF (Term Frequency-Inverse Document Frequency) vectorization technique. This method involves assigning numerical values to words based on their frequency in individual documents relative to the entire dataset. By doing so, we obtained a TF-IDF matrix, where each row represents a document, and each column corresponds to a unique word in the corpus. This matrix captures the importance of words in the context of specific documents, setting the stage for more advanced analyses.

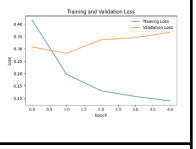
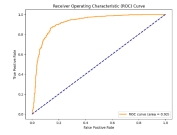
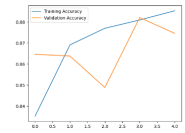
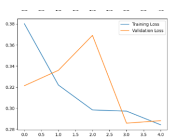
The subsequent modeling phase introduces a Bi-Directional Long Short-Term Memory (Bi-LSTM) neural network. The reason this architecture was chosen is for its ability to understand and memorize patterns in sequential data, considering the order and context of words bidirectionally. The Bi-LSTM is particularly adept at capturing dependencies in both forward and backward directions, making it a suitable choice for our text-based analysis.

In essence, the integration of these steps, from detailed preprocessing to TF-IDF vectorization and the application of a sophisticated Bi-LSTM neural network, is designed to understand the semantics and correlation present within our textual data. Each step contributes to refining our understanding of the data, ultimately enhancing the model's capability to discern and interpret the semantic content embedded in the text.

Results and Discussion

Results Table

Model Used	Accuracy	Confusion Matrix	Precision and Recall	Training and Validation Accuracy Curve	Training and Validation Loss Curve
TF-IDF					
Decision Tree	0.80	[[607 142] [159 596]]	Precision: 0.80 Recall: 0.80	-	-
Random Forest	0.88	[[667 82] [94 661]]	Precision: 0.89 Recall: 0.88	-	-
Logistic Regression	0.9015957446808 51	[612 137] [157 598]	Precision: 0.81 Recall: 0.79	-	-
Voting Classifier	0.80	[[612 137] [157 598]]	Precision: 0.81 Recall: 0.79	-	-
Recurrent Neural Network	Test Accuracy: 0.9029255509376526 Validation Accuracy: 0.88	[[678 71] [78 677]]	Precision: 0.9050802139037433 Recall: 0.8966887417218543		
Bi-Directional LSTM	Test Accuracy: 0.9102393388748169 Validation Accuracy: 0.9182	[[650 99] [60 695]]	Precision: 0.8753148614609572 Recall: 0.9205298013245033		
Convolution Neural Network	Test Accuracy: 0.8956117033958435 Validation Accuracy: 0.8956	[[676 73] [83 672]]	Precision: 0.9020134228187919 Recall: 0.8900662251655629		

Word2Vec					
Logistic Regression	**0.87**	**[[638 111] [90 665]]**	**Precision: 0.86 Recall: 0.88**	**-**	**-**
Recurrent Neural Network	**0.90**	**[[701 48] [107 648]]**	**Precision: 0.93 Recall: 0.86**		
Doc2Vec					
Logistic Regression	**0.87**	**[[639 110] [91 664]]**	**Precision: 0.86 Recall: 0.88**		
Recurrent Neural Network	**Test Accuracy: 0.8730053305625916 Validation Accuracy: 0.874**	**[[639 110] [91 664]]**	**Precision: 0.86 Recall: 0.88**		

Sample Source Code

TF-IDF Vectorization:

1. Decision Tree

```
X = df.drop(['ID', 'text', 'class', 'class_binary', 'cleaned_text', 'tokenized_text', 'lowercased_text', 'filtered_text', 'lemmatized_text', 'processed_t
y = df['class_binary']

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a decision tree classifier
clf = DecisionTreeClassifier()

# Train the classifier on the training data
clf.fit(X_train, y_train)

# Make predictions on the testing data
y_pred = clf.predict(X_test)

# Evaluate the accuracy of the classifier
accuracy = accuracy_score(y_test, y_pred)
confusion_mat = confusion_matrix(y_test, y_pred)

print(f"Accuracy: {accuracy}")
print("Confusion Matrix:\n", confusion_mat)
```

2. Random Forest

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
# Assuming 'class_binary' is the target variable
X = df.drop(['ID', 'text', 'class', 'class_binary', 'cleaned_text', 'tokenized_text', 'lowercased_text', 'filtered_text', 'lemmatized_text', 'processed_t
y = df['class_binary']

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Random Forest classifier
clf = RandomForestClassifier(n_estimators=100, random_state=42)

# Train the classifier on the training data
clf.fit(X_train, y_train)

# Make predictions on the testing data
y_pred = clf.predict(X_test)

# Evaluate the accuracy of the classifier
accuracy = accuracy_score(y_test, y_pred)
confusion_mat = confusion_matrix(y_test, y_pred)

print(f"Accuracy: {accuracy}")
print("Confusion Matrix:\n", confusion_mat)
```

3. Logistic Regression

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Assuming 'class_binary' is the target variable
X = df.drop(['ID', 'text', 'class', 'class_binary', 'cleaned_text', 'tokenized_text', 'lowercased_text', 'filtered_text', 'lemmatized_text', 'processed_t
y = df['class_binary']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize and train a logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Make predictions on the testing set
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy}")
print(f"Confusion Matrix:\n{conf_matrix}")
print(f"Classification Report:\n{classification_rep}")
```

4. Ensemble Method (Voting Classifier)

```
from sklearn.ensemble import VotingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

# Assuming 'class_binary' is the target variable
X = df.drop(['ID', 'text', 'class', 'class_binary', 'cleaned_text', 'tokenized_text', 'lowercased_text', 'filtered_text', 'lemmatized_text', 'processed_t
y = df['class_binary']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize individual models
decision_tree = DecisionTreeClassifier(random_state=42)
random_forest = RandomForestClassifier(random_state=42)
knn = KNeighborsClassifier()

# Create an ensemble model using a soft voting strategy
ensemble_model = VotingClassifier(
    estimators=[('decision_tree', decision_tree), ('random_forest', random_forest), ('knn', knn)],
    voting='soft'  # Soft voting takes the average of probabilities
)

# Train the ensemble model
ensemble_model.fit(X_train, y_train)

# Make predictions on the testing set
y_pred_ensemble = ensemble_model.predict(X_test)

# Evaluate the ensemble model
accuracy_ensemble = accuracy_score(y_test, y_pred_ensemble)
conf_matrix_ensemble = confusion_matrix(y_test, y_pred_ensemble)
classification_rep_ensemble = classification_report(y_test, y_pred_ensemble)

print(f"Ensemble Model Accuracy: {accuracy_ensemble}")
print(f"Ensemble Model Confusion Matrix:\n{conf_matrix_ensemble}")
print(f"Ensemble Model Classification Report:\n{classification_rep_ensemble}")
```

5. Recurrent Neural Network

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import LabelEncoder
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, SpatialDropout1D
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
from tensorflow.keras.callbacks import EarlyStopping
X_train, X_test, y_train, y_test = train_test_split(df['processed_text'], df['class_binary'], test_size=0.2, random_state=42)
max_words = 5000  # Maximum number of words to keep
max_len = 100  # Maximum length of sequences
tokenizer = Tokenizer(num_words=max_words, split=' ')
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

X_train_pad = pad_sequences(X_train_seq, maxlen=max_len)
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len)

# Encode labels
label_encoder = LabelEncoder()
y_train_encoded = label_encoder.fit_transform(y_train)
y_test_encoded = label_encoder.transform(y_test)

# Build the RNN model
embedding_dim = 50  # Dimension of word embeddings
model = Sequential()
model.add(Embedding(input_dim=max_words, output_dim=embedding_dim, input_length=max_len))
model.add(SpatialDropout1D(0.2))
model.add(LSTM(80))
model.add(Dense(1, activation='sigmoid'))

# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
# Train the model
epochs = 5
batch_size = 32
#model.fit(X_train_pad, y_train_encoded, epochs=epochs, batch_size=batch_size, validation_data=(X_test_pad, y_test_encoded))
history = model.fit(X_train_pad, y_train_encoded, epochs=epochs, batch_size=batch_size,
                    validation_data=(X_test_pad, y_test_encoded), callbacks=[early_stopping])
# Evaluate the model on the test set
loss, accuracy = model.evaluate(X_test_pad, y_test_encoded)
print(f"Test Accuracy: {accuracy}")
```

6. Bi- Directional LSTM Neural Network

```
from tensorflow.keras.layers import Bidirectional
X_train, X_test, y_train, y_test = train_test_split(df['processed_text'], df['class_binary'], test_size=0.2, random_state=42)
max_words = 5000  # Maximum number of words to keep
max_len = 100  # Maximum length of sequences
tokenizer = Tokenizer(num_words=max_words, split=' ')
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

X_train_pad = pad_sequences(X_train_seq, maxlen=max_len)
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len)

# Encode labels
label_encoder = LabelEncoder()
y_train_encoded = label_encoder.fit_transform(y_train)
y_test_encoded = label_encoder.transform(y_test)

# Build the Bidirectional LSTM model
model = Sequential()
model.add(Embedding(input_dim=max_words, output_dim=embedding_dim, input_length=max_len))
model.add(SpatialDropout1D(0.5))
model.add(Bidirectional(LSTM(80, dropout=0.5)))
model.add(Dense(1, activation='sigmoid'))

# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

# Train the model
epochs = 5
batch_size = 32
history = model.fit(X_train_pad, y_train_encoded, epochs=epochs, batch_size=batch_size,
                    validation_data=(X_test_pad, y_test_encoded), callbacks=[early_stopping])

# Evaluate the model on the test set
loss, accuracy = model.evaluate(X_test_pad, y_test_encoded)
print(f"Test Accuracy: {accuracy}")
```

7. Convolution Neural Network

```
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense, SpatialDropout1D
from tensorflow.keras.layers import Dropout
X_train, X_test, y_train, y_test = train_test_split(df['processed_text'], df['class_binary'], test_size=0.2, random_state=42)

max_words = 5000  # Maximum number of words to keep
max_len = 100  # Maximum length of sequences

tokenizer = Tokenizer(num_words=max_words, split=' ')
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

X_train_pad = pad_sequences(X_train_seq, maxlen=max_len)
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len)

# Encode labels
label_encoder = LabelEncoder()
y_train_encoded = label_encoder.fit_transform(y_train)
y_test_encoded = label_encoder.transform(y_test)

# Build the CNN model
embedding_dim = 100  # Dimension of word embeddings
model = Sequential()
model.add(Embedding(input_dim=max_words, output_dim=embedding_dim, input_length=max_len))
model.add(Conv1D(128, 5, activation='relu'))
model.add(GlobalMaxPooling1D())
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))

# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

# Train the model
epochs = 5
batch_size = 32
history = model.fit(X_train_pad, y_train_encoded, epochs=epochs, batch_size=batch_size,
                    validation_data=(X_test_pad, y_test_encoded), callbacks=[early_stopping])

# Evaluate the model on the test set
loss, accuracy = model.evaluate(X_test_pad, y_test_encoded)
print(f"Test Accuracy: {accuracy}")
```

Word2Vec:

8. Logistic Regression

Creating a model using logistic regression

[53]:
```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(df['word2vec_features'].to_list(), df['class_binary'], test_size=0.2, random_state=42)

# Create a logistic regression model, after applying gridsearch, the best value for C and max_iter have been found to be 100 and 500
logreg_model = LogisticRegression(C=100, max_iter=500)

# Fit the model
logreg_model.fit(X_train, y_train)
```

```
C:\Users\Admin\nidcatpythonlab\Lib\site-packages\sklearn\linear_model\_logistic.py:460: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

[53]:
```
LogisticRegression
LogisticRegression(C=100, max_iter=500)
```

[54]:
```
predictions = logreg_model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, predictions)
print(f'Accuracy: {accuracy:.2f}')
```

```
Accuracy: 0.87
```

[56]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions)
recall = recall_score(y_test, predictions)
f1 = f1_score(y_test, predictions)
conf_matrix = confusion_matrix(y_test, predictions)

print(f'Accuracy: {accuracy:.2f}')
print(f'Precision: {precision:.2f}')
print(f'Recall: {recall:.2f}')
print(f'F1 Score: {f1:.2f}')
print('Confusion Matrix:')
print(conf_matrix)
```

```
Accuracy: 0.87
Precision: 0.86
Recall: 0.88
F1 Score: 0.87
Confusion Matrix:
[[638 111]
 [ 90 665]]
```

9. Recurrent Neural Network

Creating a model using a Recurrent neural network

```
from keras.models import Sequential
from keras.layers import Embedding, LSTM, Dense
from keras.preprocessing.text import Tokenizer
from keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Tokenize and pad the text data
max_words = 5000  # Maximum number of words to keep
max_len = 100  # Maximum length of sequences
tokenizer = Tokenizer(num_words=max_words, split=' ')
tokenizer.fit_on_texts(df['lemmatized_text'])
sequences = tokenizer.texts_to_sequences(df['lemmatized_text'])
X = pad_sequences(sequences, maxlen=100)
```

```
from tensorflow.keras.callbacks import EarlyStopping
from keras.layers import Dropout
X_train, X_test, y_train, y_test = train_test_split(X, df['class_binary'], test_size=0.2, random_state=42)

# Create the RNN model
model = Sequential()
model.add(Embedding(input_dim=len(tokenizer.word_index) + 1, output_dim=100, input_length=100))
model.add(LSTM(100))  # You can adjust the number of LSTM units
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
import matplotlib.pyplot as plt
# Train the model and record the history
history = model.fit(X_train, y_train, epochs=5, batch_size=32, validation_split=0.2, callbacks=[early_stopping])

# Extract training history
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

# Plot training and validation loss
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(train_loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

# Plot training and validation accuracy
plt.subplot(1, 2, 2)
plt.plot(train_acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()
```

Doc2Vec:

10. Logistic Regression

Applying Logistic Regression

```
X_train, X_test, y_train, y_test = train_test_split(
    np.vstack(df['doc_vector']), df['class_binary'], test_size=0.2, random_state=42
)

# Logistic Regression
logreg = LogisticRegression()
logreg.fit(X_train, y_train)

# Predictions
y_pred = logreg.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
```

11. Recurrent Neural Network

Applying a Recurrent Neural Network

```
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, SpatialDropout1D
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Function to get sequences from Word2Vec vectors
def get_sequences(tokens, model):
    sequences = [model.wv[word] for word in tokens if word in model.wv.index_to_key]
    return sequences

# Get sequences
df['word2vec_sequences'] = df['lemmatized_text'].apply(lambda x: get_sequences(x, model_w2v))

max_len = 50
# Pad sequences to a fixed length
X_word2vec = pad_sequences(df['word2vec_sequences'], maxlen=max_len, dtype='float32')

# Split the data into training and testing sets
X_train_word2vec, X_test_word2vec, y_train_word2vec, y_test_word2vec = train_test_split(
    X_word2vec, df['class_binary'], test_size=0.2, random_state=42
)

# Build the RNN model
embedding_dim = 100  # Dimension of word embeddings
model = Sequential()
model.add(LSTM(80, input_shape=(max_len, embedding_dim)))
model.add(Dense(1, activation='sigmoid'))

# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

# Train the model
epochs = 5
batch_size = 32
model.fit(X_train_word2vec, y_train_word2vec, epochs=epochs, batch_size=batch_size, validation_split=0.2)

# Evaluate the model on the test set
loss, accuracy = model.evaluate(X_test_word2vec, y_test_word2vec)
print(f"Test Accuracy: {accuracy}")
```

Discussion

Through the variety of models used, it can be concluded that using the TF-IDF vectorization algorithm with a Bi-directional LSTM gives the highest accuracy and is the most precise.

The TF-IDF vectorization algorithm, when coupled with a Bi-directional Long Short-Term Memory (Bi-LSTM) neural network, emerges as the optimal choice for several reasons. Firstly, the TF-IDF approach excels at capturing the significance of words within documents, offering a nuanced representation of the data that considers both term frequency and document specificity. This is particularly crucial in textual data analysis where understanding the relative importance of words enhances the model's ability to discern context and meaning.

The Bi-directional LSTM, chosen for its unique architecture capable of capturing dependencies in both forward and backward directions, is well-suited for sequential data. This is particularly advantageous in our case, where the order and context of words play a vital role in determining meaning. The Bi-LSTM excels at grasping long-range dependencies and is effective in modeling sequential patterns, making it a strong candidate for text data analysis.

Moreover, the ensemble models such as Decision Tree, Random Forest, and the Voting Classifier, while powerful in certain contexts, may not capture the intricate sequential relationships inherent in textual data as effectively as the Bi-LSTM. The Convolutional Neural Network (CNN), designed for image recognition, might not be the optimal choice for our text-based task, as it focuses on detecting hierarchical patterns in spatial data rather than capturing the sequential nature of language.

In contrast, logistic regression, while straightforward and interpretable, may lack the capacity to capture complex sequential dependencies present in textual data, making it less suitable for tasks where context and order matter.

In summary, the choice of the TF-IDF vectorization algorithm with a Bi-directional LSTM is rooted in its ability to effectively capture the intricate patterns and dependencies within textual data. While each model serves a purpose, the TF-IDF with Bi-LSTM combination excels in the nuanced analysis of sequential and context-dependent information, leading to higher accuracy and precision in our specific task. The comprehensive consideration of both word importance and sequential relationships distinguishes this chosen model as the most adept in extracting meaningful insights from the data at hand.

Conclusion

In order to meet the urgent demand for preventative mental health care, our project uses natural language processing (NLP) techniques to identify indications of suicidal ideation and mental suffering within digital text data. The programme uses TF-IDF analysis to identify persons who may be at danger by identifying not only explicit allusions but also subtle language patterns that may indicate emotional distress. This novel method closes gaps in mental health care by enabling early detection and intervention. It recognises mental health difficulties that go beyond suicidal thoughts, demonstrating how NLP may be used to diagnose a wide range of conditions, including chronic anxiety and borderline personality disorder. In the end, this merging of technology and empathy represents a revolutionary move towards a proactive and compassionate mental health environment that seeks to save lives and provide prompt assistance to those in need.

References

[1] Priyamvada, B., Singhal, S., Nayyar, A. et al. Stacked CNN - LSTM approach for prediction of suicidal ideation on social media. Multimed Tools Appl 82, 27883–27904 (2023). https://doi.org/10.1007/s11042-023-14431-z

[2] Aldhyani, T.H.H., Alsubari, S.N., Alshebami, A.S., Alkahtani, H., & Ahmed, Z.A.T. (2022). Detecting and Analyzing Suicidal Ideation on Social Media Using Deep Learning and Machine Learning Models. International Journal of Environmental Research and Public Health, 19(19), 12635. doi: 10.3390/ijerph191912635. PMID: 36231935; PMCID: PMC9565132.

[3] Ji, S., Pan, S., Li, X., Cambria, E., Long, G., & Huang, Z. (2021). Suicidal Ideation Detection: A Review of Machine Learning Methods and Applications. IEEE Transactions on Computational Social Systems, 8(1), 214-226. doi: 10.1109/TCSS.2020.3021467.

[4] Liu, J., Shi, M., & Jiang, H. (2022). Detecting Suicidal Ideation in Social Media: An Ensemble Method Based on Feature Fusion. International Journal of Environmental Research and Public Health, 19(13), 8197. doi: 10.3390/ijerph19138197. PMID: 35805856; PMCID: PMC9266694.

[5] Chatterjee, M., Kumar, P., Samanta, P., & Sarkar, D. (2022). Suicide ideation detection from online social media: A multi-modal feature-based technique. International Journal of Information Management Data Insights, 2(2), 100103. ISSN 2667-0968.

[6] Young, J., Bishop, S., Humphrey, C., & Pavlacic, J. M. (2023). A review of natural language processing in the identification of suicidal behavior. Journal of Affective Disorders Reports, 12, 100507. ISSN 2666-9153. doi: 10.1016/j.jadr.2023.100507.

[7] Haque R, Islam N, Islam M, Ahsan MM. A Comparative Analysis on Suicidal Ideation Detection Using NLP, Machine, and Deep Learning. Technologies. 2022; 10(3):57. https://doi.org/10.3390/technologies10030057

[8] Zhang, T., Schoene, A.M., Ji, S. et al. Natural language processing applied to mental illness detection: a narrative review. npj Digit. Med. 5, 46 (2022). https://doi.org/10.1038/s41746-022-00589-7

[9] Ji, S., Li, X., Huang, Z., and Cambria, E. (2021). 'Suicidal ideation and mental disorder detection with attentive relation networks'. Neural Computing and Applications, 34(13), pp. 10309–10319.

[10] Basyouni, A., Abdelkader, H. and Ali, A., 2023. A New Approach To Suicide Ideation Detection from Text Content. IJCI. International Journal of Computers and Information, 10(2), pp.1-15.

[11] Bhaumik, R., Srivastava, V., Jalali, A., Ghosh, S. and Chandrasekaran, R., 2023. MindWatch: A Smart Cloud-based AI solution for Suicide Ideation Detection leveraging Large Language Models. medRxiv, pp.2023-09.

[12] Lamichhane, B., 2023. Evaluation of chatgpt for nlp-based mental health applications. arXiv preprint arXiv:2303.15727.

[13] NLP-UNED at eRisk 2021: self-harm early risk detection with TF-IDF and linguistic features : Elena Campillo-Ageitos1 , Hermenegildo Fabregat1 , Lourdes Araujo1,2 and Juan Martinez-Romo1,2
https://ceur-ws.org/Vol-2936/paper-78.pdf

[14] Detection of self-harm and suicidal ideation in emergency department triage notes : Vlada Rozova, Katrina Witt, Jo Robinson, Yan Li, Karin Verspoor
https://academic.oup.com/jamia/article-abstract/29/3/472/6460149

[15] Detecting suicide risk using knowledge-aware natural language processing and counseling service data
https://www.sciencedirect.com/science/article/abs/pii/S0277953621005086

[16] Machine Learning to Differentiate Risk of Suicide Attempt and Self-harm After General Medical Hospitalization of Women With Mental Illness
https://journals.lww.com/lww-medicalcare/abstract/2021/02001/machine_learning_to_differentiate_risk_of_suicide.14.aspx?context=latestarticles

[17] Portability of natural language processing methods to detect suicidality from clinical text in US and UK electronic health records
https://www.sciencedirect.com/science/article/pii/S2666915322001226

[18] Application of Natural Language Processing (NLP) in Detecting and Preventing Suicide Ideation: A Systematic Review
https://www.mdpi.com/1660-4601/20/2/1514

[19] 7. A review of natural language processing in the identification of suicidal behavior
https://doi.org/10.1016/j.jadr.2023.100507

[20]Tope Oyelade, Abayomi Arowosegbe.,2023 Application of Natural Language Processing (NLP) in Detecting and Preventing Suicide Ideation: A Systematic Review
https://www.mdpi.com/1660-4601/20/2/1514

[21]Julie M Kafka, Mike D Fliss, Pamela J Trangenstein, Luz McNaughton Reyes, Brian W Pence, Kathryn E Moracco, Detecting intimate partner violence circumstance for suicide: development and validation of a tool using natural language processing and supervised machine learning in the National Violent Death Reporting System
https://injuryprevention.bmj.com/content/29/2/134.abstract

[22] Zhongzhi Xu , Yucan Xu , Florence Cheung , Mabel Cheng , Daniel Lung , Yik Wa Law , Byron Chiang, Qingpeng Zhang , Paul S.F. Yip, 2021, Detecting suicide risk using knowledge-aware natural language processing and counseling service data
https://www.sciencedirect.com/science/article/abs/pii/S0277953621005086

[23] Pratyaksh Jain, Karthik Ram Srinivas and Abhishek Vichare, Depression and Suicide Analysis Using Machine Learning and NLP
https://iopscience.iop.org/article/10.1088/1742-6596/2161/1/012034/meta

[24] Anshu Malhotra, Rajni Jindal. 2022, Deep learning techniques for suicide and depression detection from online social media: A scoping review
https://www.sciencedirect.com/science/article/abs/pii/S1568494622007621

[25] Harnain Kour & Manoj Kumar Gupta, 2022, Depression and Suicide Prediction Using Natural Language Processing and Machine Learning
https://link.springer.com/chapter/10.1007/978-981-16-8664-1_11

[26] Identification Method of Psychological and Behavioral Characteristics of Adolescent Implicit Suicide Based on Distributed Data Acquisition Model.

H. Luo, "Identification Method of Psychological and Behavioral Characteristics of Adolescent Implicit Suicide Based on Distributed Data Acquisition Model," *2022 Euro-Asia Conference on Frontiers of Computer Science and Information Technology (FCSIT)*, Beijing, China, 2022, pp. 149-158, doi: 10.1109/FCSIT57414.2022.00039.

https://ieeexplore-ieee-org.egateway.vit.ac.in/document/10098249

[27]Machine Learning based Suicide Prediction

B. Srinu, P. N. L. Bhavana, B. Tarun Reddy and B. Vaishnavi, "Machine Learning based Suicide Prediction," *2022 6th International Conference on Computing Methodologies and Communication (ICCMC)*, Erode, India, 2022, pp. 953-957, doi: 10.1109/ICCMC53470.2022.9754035.
https://ieeexplore-ieee-org.egateway.vit.ac.in/document/9754035

[28]Social Media Mining For Suicidal Ideation

S. Satya and R. Jindal, "Social Media Mining For Suicidal Ideation," *2022 4th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)*, Greater Noida, India, 2022, pp. 1466-1471, doi: 10.1109/ICAC3N56670.2022.10073980.
https://ieeexplore-ieee-org.egateway.vit.ac.in/document/10073980

[29] Techniques for Suicidal Ideation Prediction: a Qualitative Systematic Review

M. Beriwal and S. Agrawal, "Techniques for Suicidal Ideation Prediction: a Qualitative Systematic Review," *2021 International Conference on INnovations in Intelligent SysTems and Applications (INISTA)*, Kocaeli, Turkey, 2021, pp. 1-8, doi: 10.1109/INISTA52262.2021.9548444.

https://ieeexplore-ieee-org.egateway.vit.ac.in/document/9548444

[30]Knowledge-Infused Dynamic Embedding for Predicting the Severity of Suicidal Ideation

D. M. Lee and H. Moradi, "Knowledge-Infused Dynamic Embedding for Predicting the Severity of Suicidal Ideation in Social Media," *2022 International Conference on Computational Science and Computational Intelligence (CSCI)*, Las Vegas, NV, USA, 2022, pp. 760-766, doi: 10.1109/CSCI58124.2022.00139.

https://ieeexplore-ieee-org.egateway.vit.ac.in/document/10216651

[31] Detection of Suicide Ideation in Twitter using ANN

K. Y. D. H. T. Yatapala and B. T. G. S. Kumara, "Detection of Suicide Ideation in Twitter using ANN," 2021 6th International Conference on Information Technology Research (ICITR), Moratuwa, Sri Lanka, 2021, pp. 1-5, doi: 10.1109/ICITR54349.2021.9657404.

https://ieeexplore-ieee-org.egateway.vit.ac.in/document/9657404

[32] A Novel Optimizer Technique for Suicide Prediction In Twitter Environment

A. A. Rezig, "A Novel Optimizer Technique for Suicide Prediction In Twitter Environment," *2021 International Conference on Information Systems and Advanced Technologies (ICISAT)*, Tebessa, Algeria, 2021, pp. 1-5, doi: 10.1109/ICISAT54145.2021.9678419.

https://ieeexplore-ieee-org.egateway.vit.ac.in/document/9678419